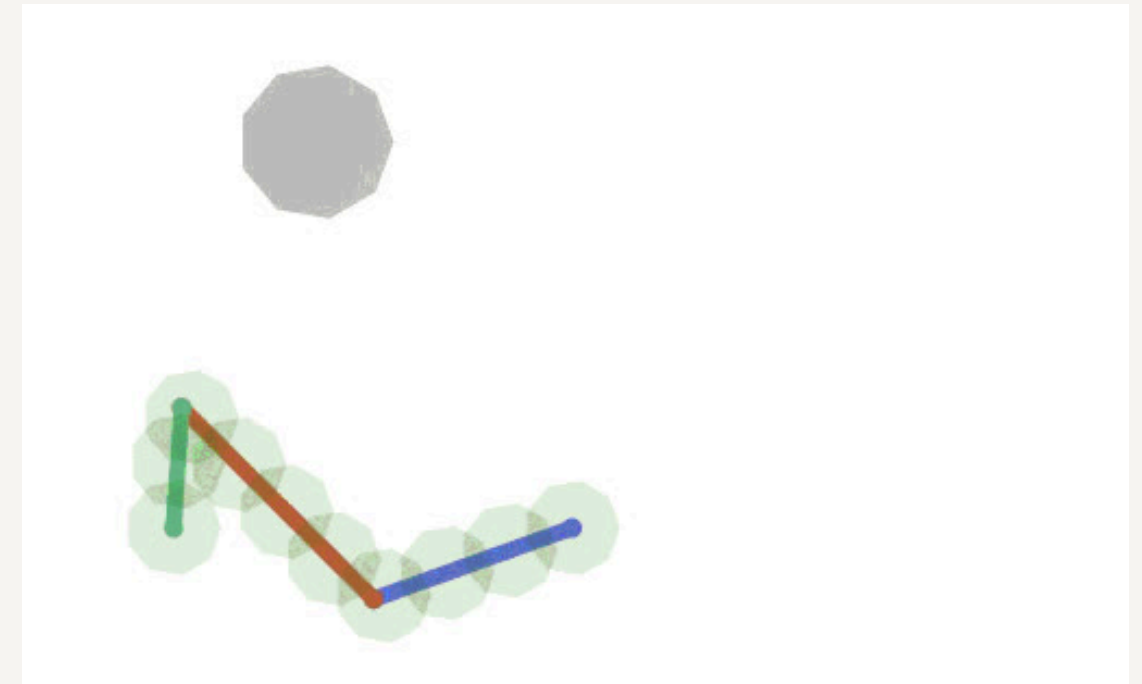


Warm-starting a Motion Planner using Self-Supervised Learning



28.05.2026

Timo Matuszewski, Gabriel Salinas

Topic #10 (Project ID 4) | Advanced Deep Learning for Robotics

Topic and Motivation

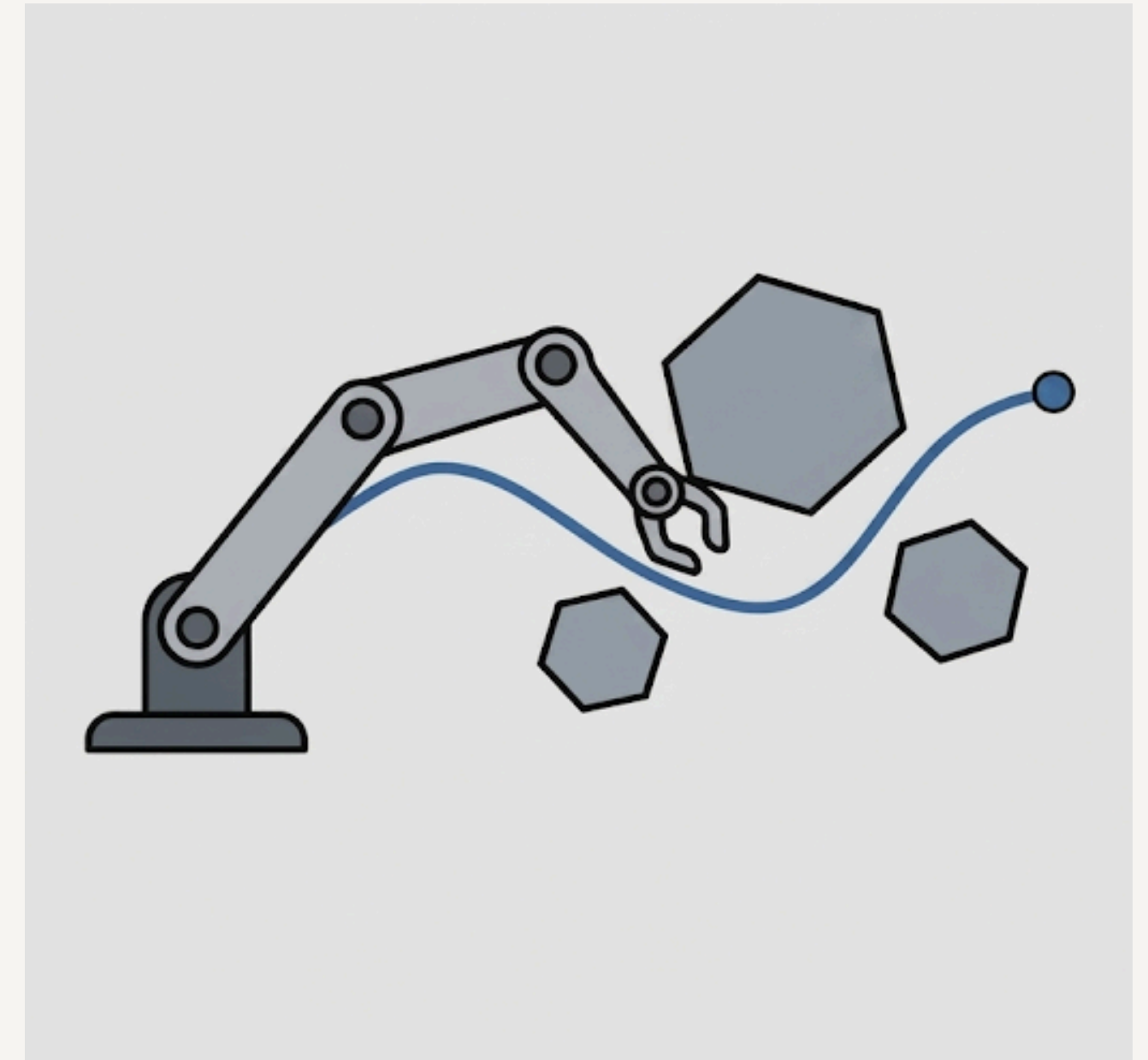
Problem Context

Motion planning for 2D robotic arms requires smooth and collision-free trajectories. Classical optimization methods depend heavily on good initialization, so poor starting points cause slow convergence and entrapment in local minima.

Approach proposed: Generate intelligent warm-start trajectories to overcome these limitations.

Research Goals

- Combine Deep Learning with classical trajectory optimization
- Predict near-feasible trajectories prior to optimization
- Improve collision avoidance, smoothness and convergence
- Enhance robustness across diverse scenarios
- Reduce dependence on global search methods
- Evaluate performance against naive initialization baselines



AI generated image for illustration

Methods Used and What Has Been Done So Far

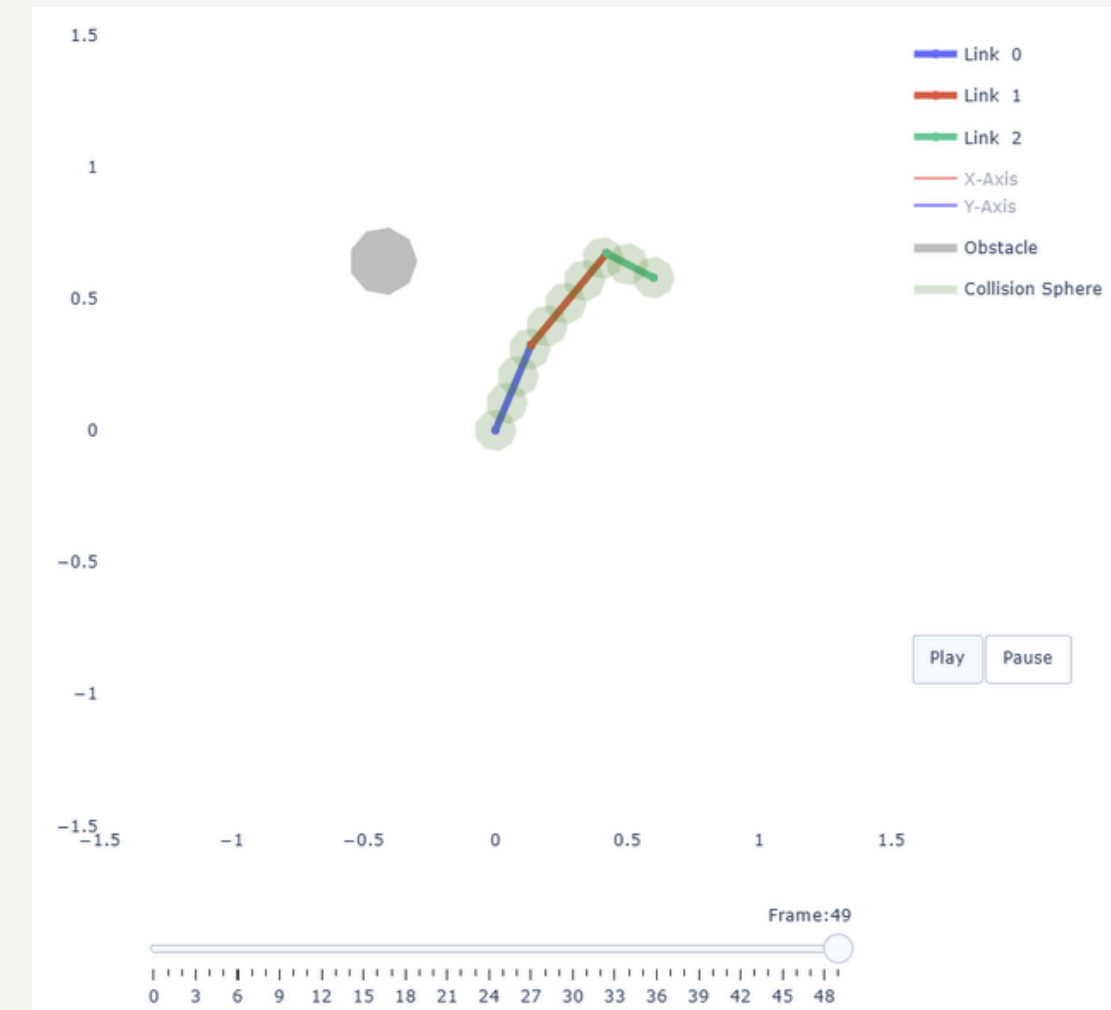
- ▶ Self-supervised learning framework operates without labeled trajectories, enabling scalable training data generation.
- ▶ Developed local libraries for data generation, losses functions, models design, training, utils and visualization.

```
data.py  training.py
losses.py  utils.py
models.py  visualization.py
```

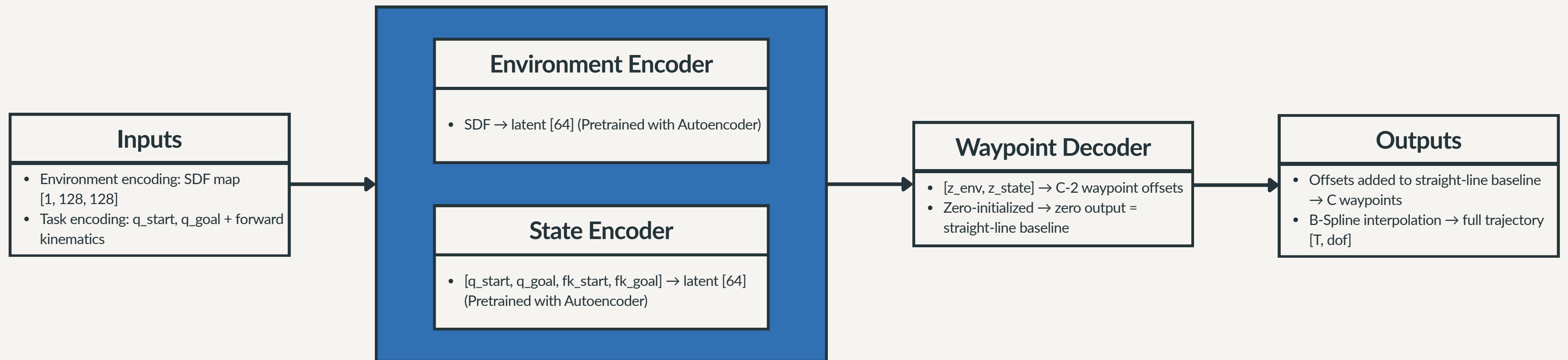
- ▶ Loss function components: collision, smoothness, spacing, exploration costs and joint limits violation penalty.

$$\begin{aligned} & C_{\text{coll}}(p(q_i, b), \text{sdf}) \mid C_{\text{smooth}}(\tau, \delta t) \mid C_{\text{space}}(\tau) \\ & C_{\text{exp}}(\tau, q_s, q_g, \epsilon) \mid C_{\text{limits}}(q_i, q_{\min}, q_{\max}) \\ & \downarrow \\ & C(\tau) = \sum_{i=1}^N \left(\sum_{b \in B} w_1 C_{\text{coll}}(p(q_i, b), \text{sdf}) + w_2 C_{\text{limits}}(q_i, q_{\min}, q_{\max}) \right) \\ & \quad + w_3 C_{\text{smooth}}(\tau, \delta t) + w_4 C_{\text{space}}(\tau) + w_5 C_{\text{exp}}(\tau, q_s, q_g, \epsilon) \end{aligned}$$

- ▶ Designed an initial architecture using pre-trained environment and state autoencoders.
- ▶ Utilized SimpleArm repository by the Tutor Lennard Stoellger for basic functions, simulation and visualization of 2D robotic arm environments.
- ▶ Experiments conducted in a 2D robotic arm simulation with randomly generated obstacle configurations.

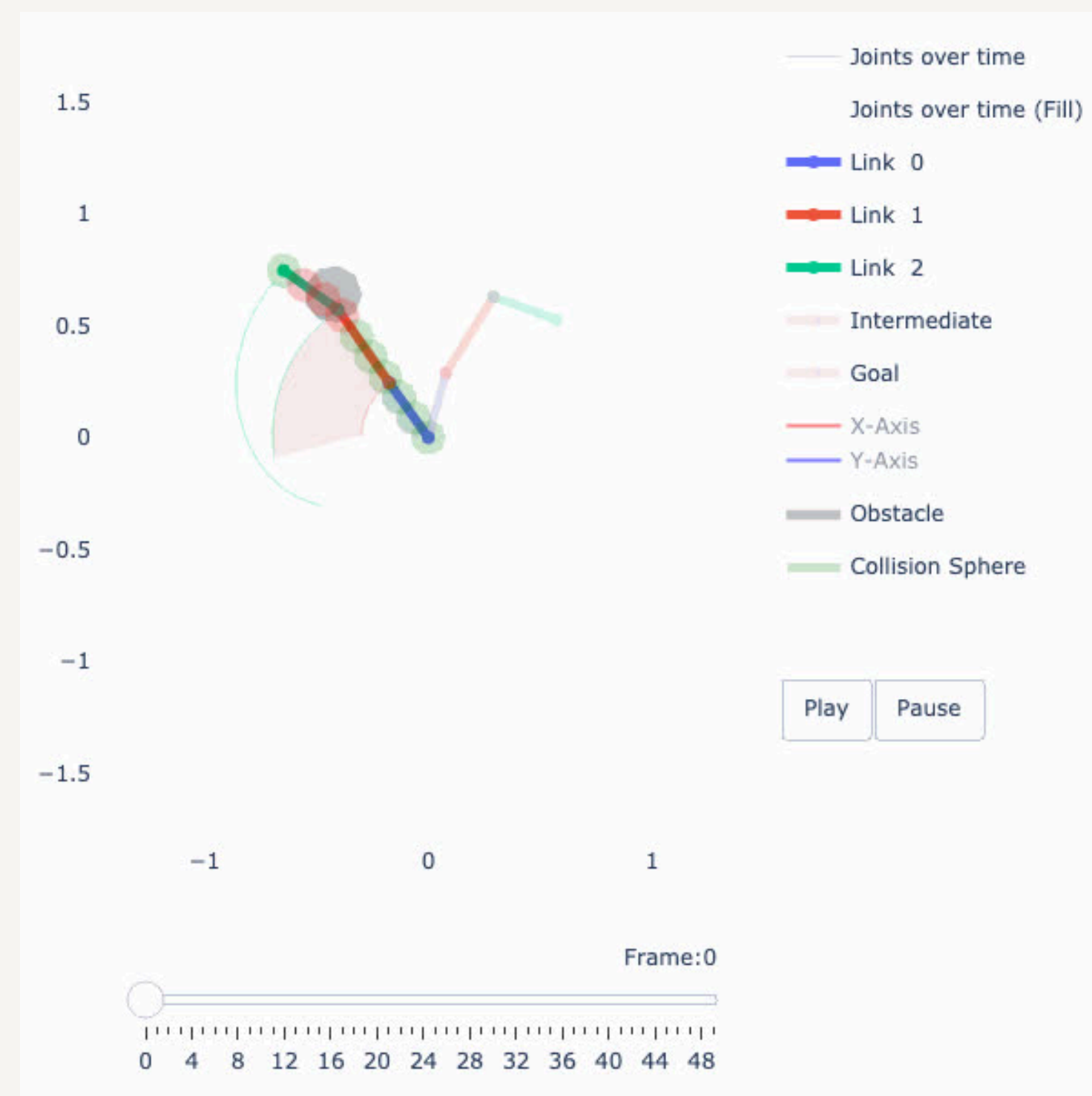
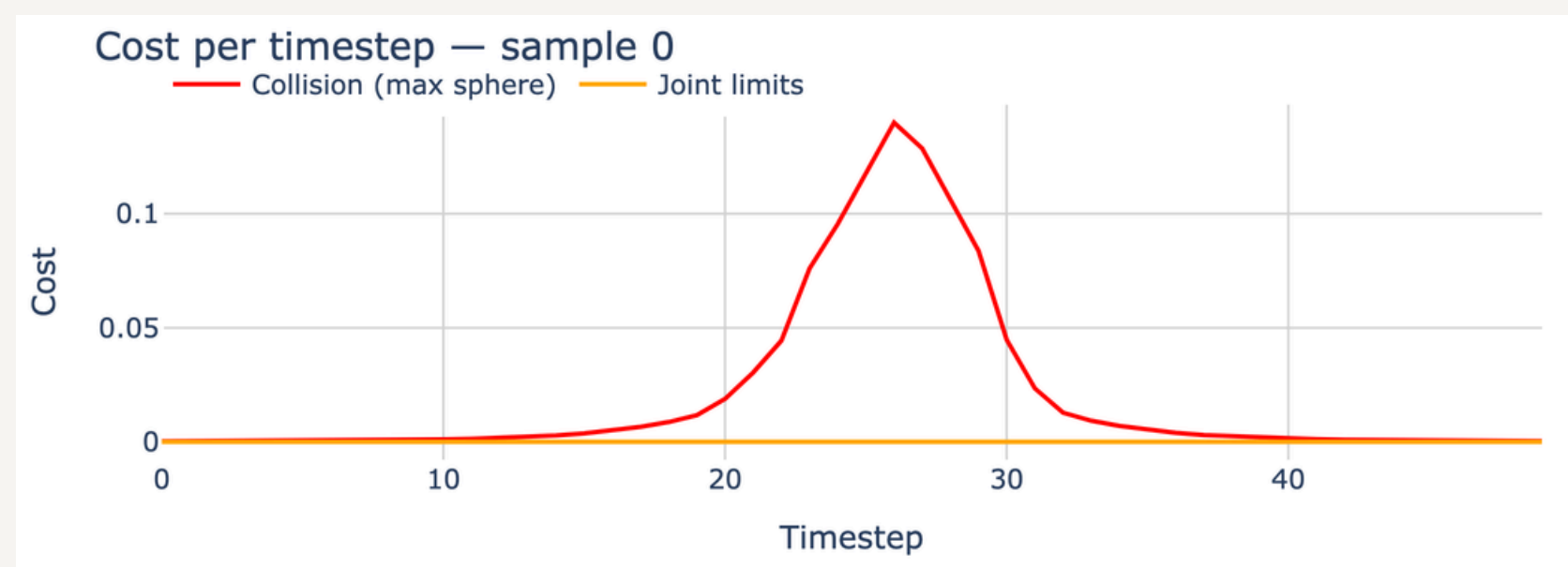


Methods: Architecture Details



What if the network outputs offsets equal to zero?

- The straight-line is not adapted to reduce the cost functions.
- The B-spline interpolation outputs a simple circular arc from the start to the goal positions.
- The arm follows the easiest trajectory as if there were no obstacles.



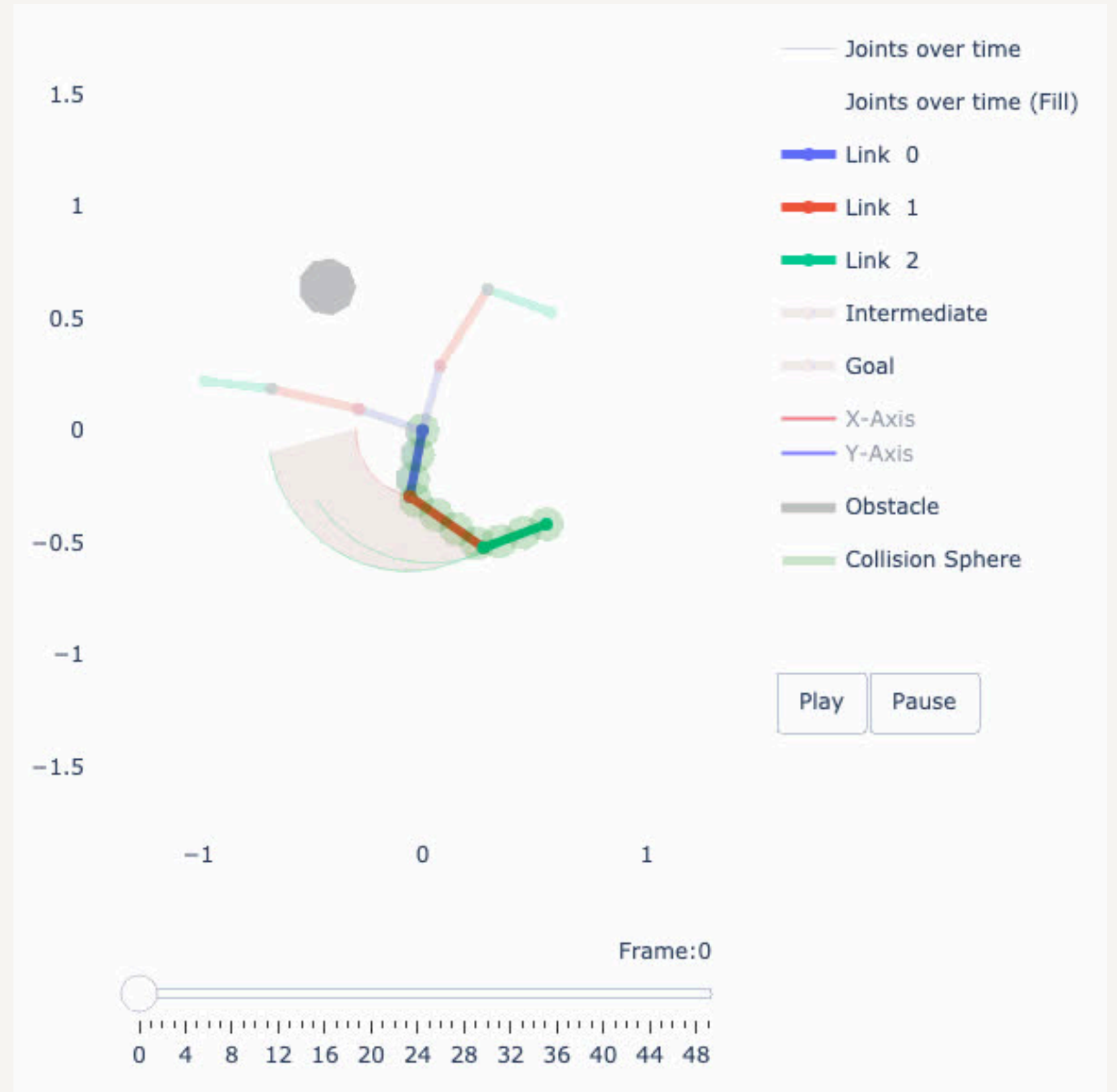
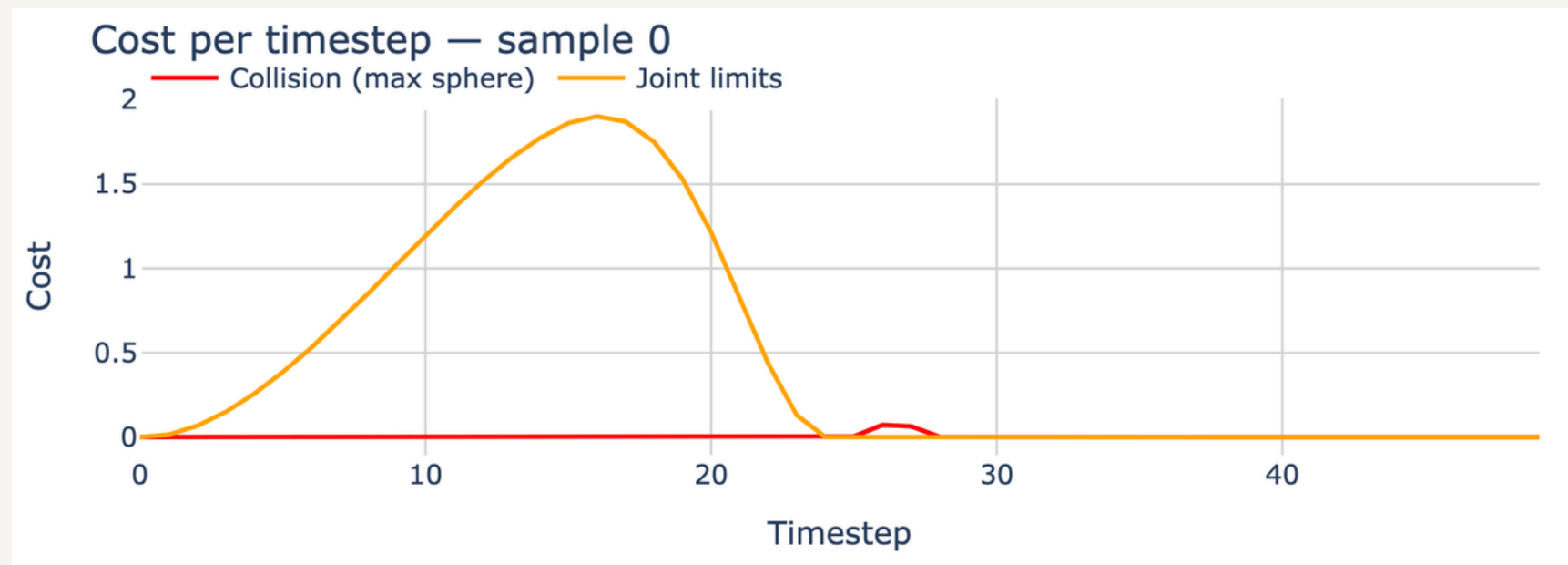
Experiments Evolution - 01

Overfitting on one sample:

- Joint-limits cost
- Trajectory-smoothness cost
- Mean collision cost of every trajectory position of the robot

Problem:

- Speeds up to minimize collision cost.
- “Tunneling” → no trajectory point lies on the obstacle → 0 collision-loss



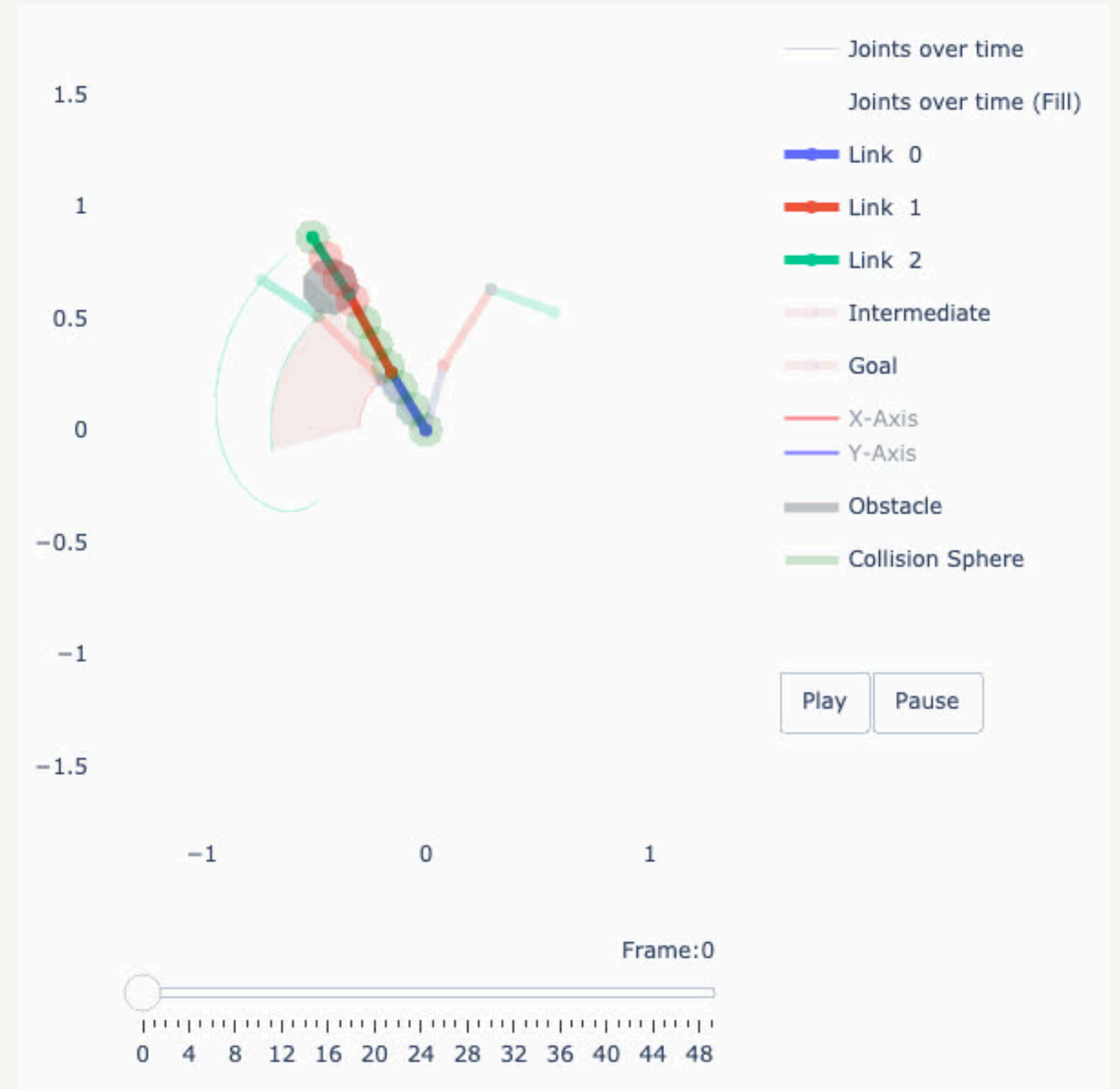
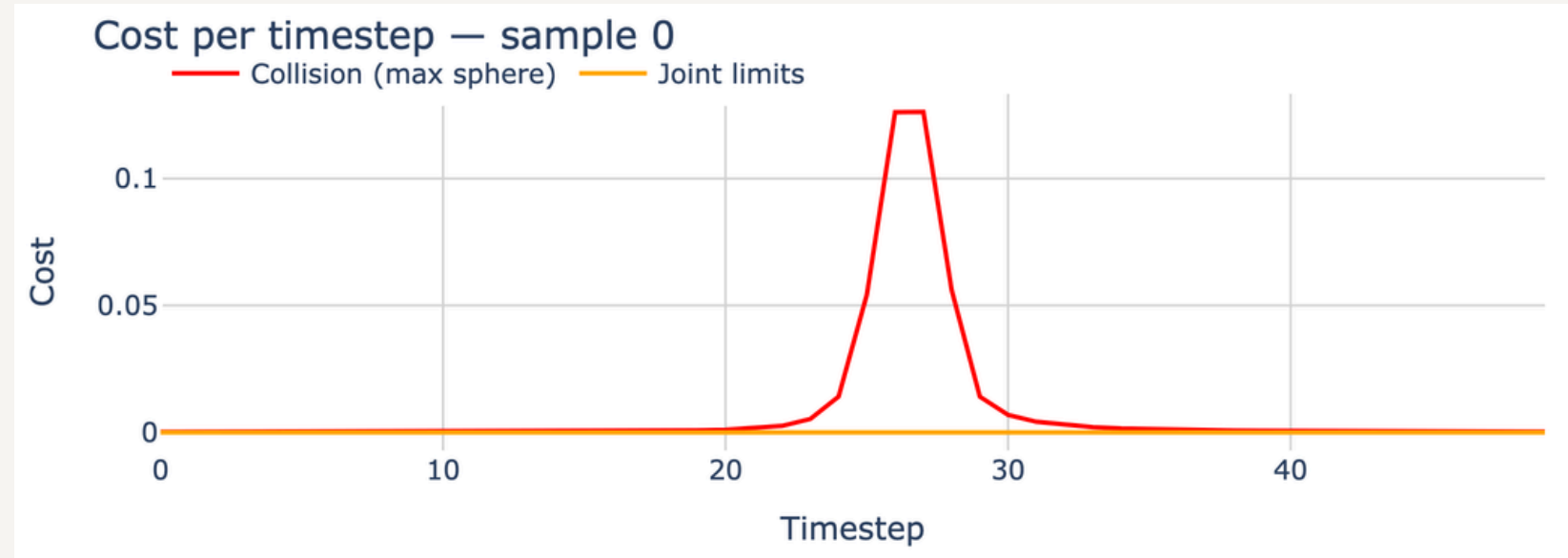
Experiments Evolution - 02

Possible fixes:

1. Collision cost based on worst-case trajectory position only → Only this cost is taken as collision cost for the whole trajectory.
2. Robot links approximated as spheres:
 - Spheres are inflated by half their travel distance per timestep → Prevents tunneling

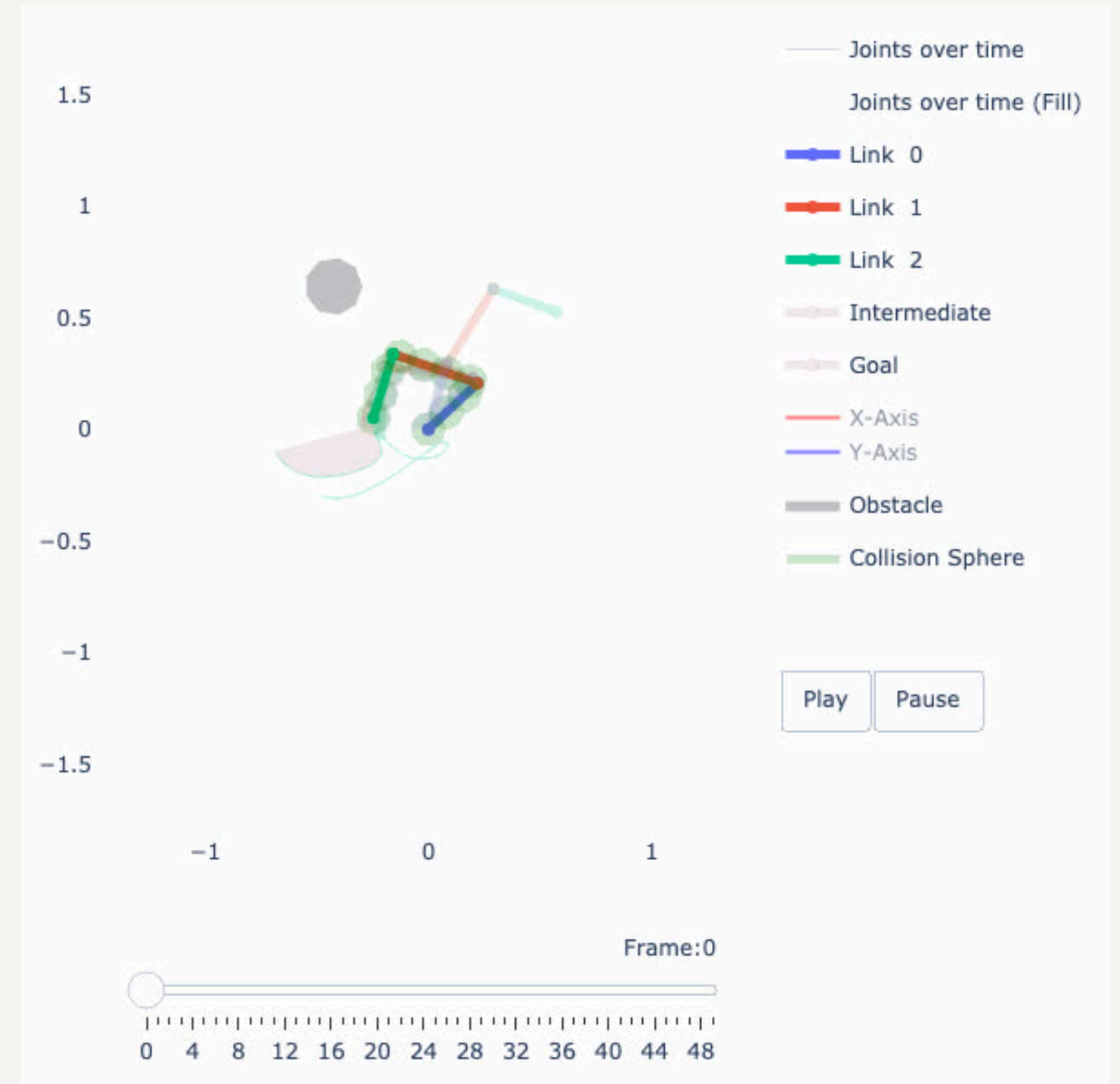
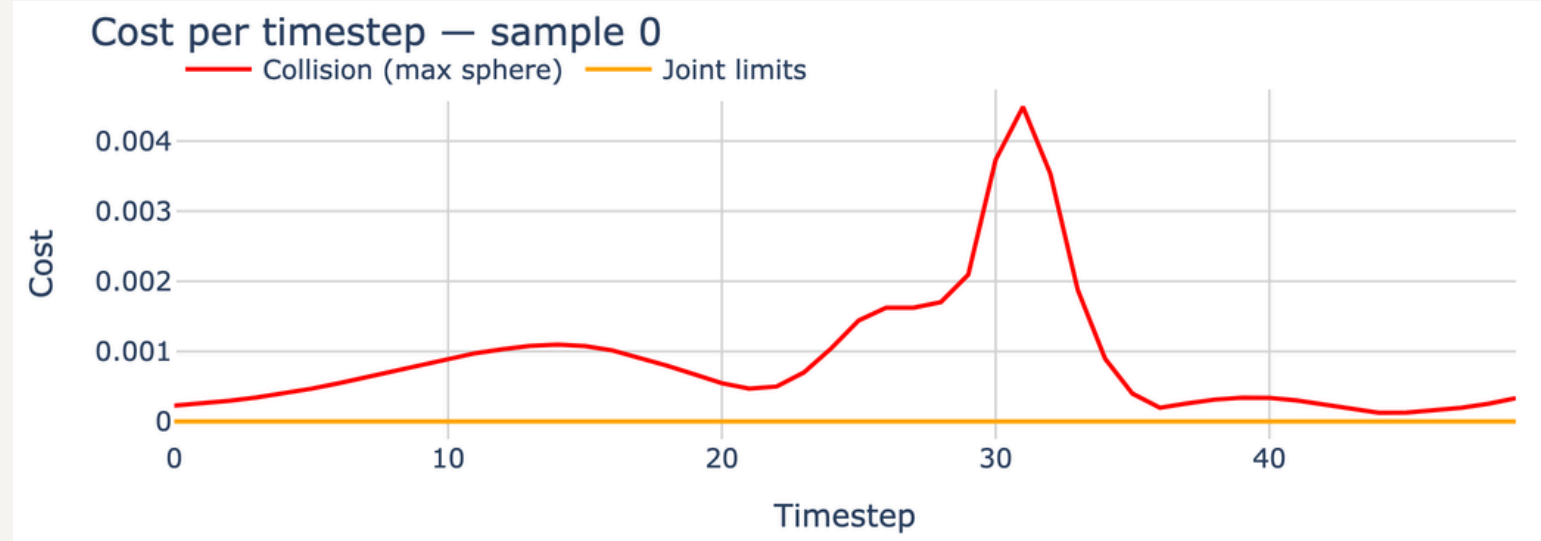
Problem:

- Initializes at straight-line interpolation baseline.
- Doesn't escape local minima.



Experiments Evolution - 03

- Fix: Added exploration cost
 - Compute deviation from straight-line baseline
 - Small deviation + collision → high cost
 - Encourages larger offsets/rerouting from initial trajectory

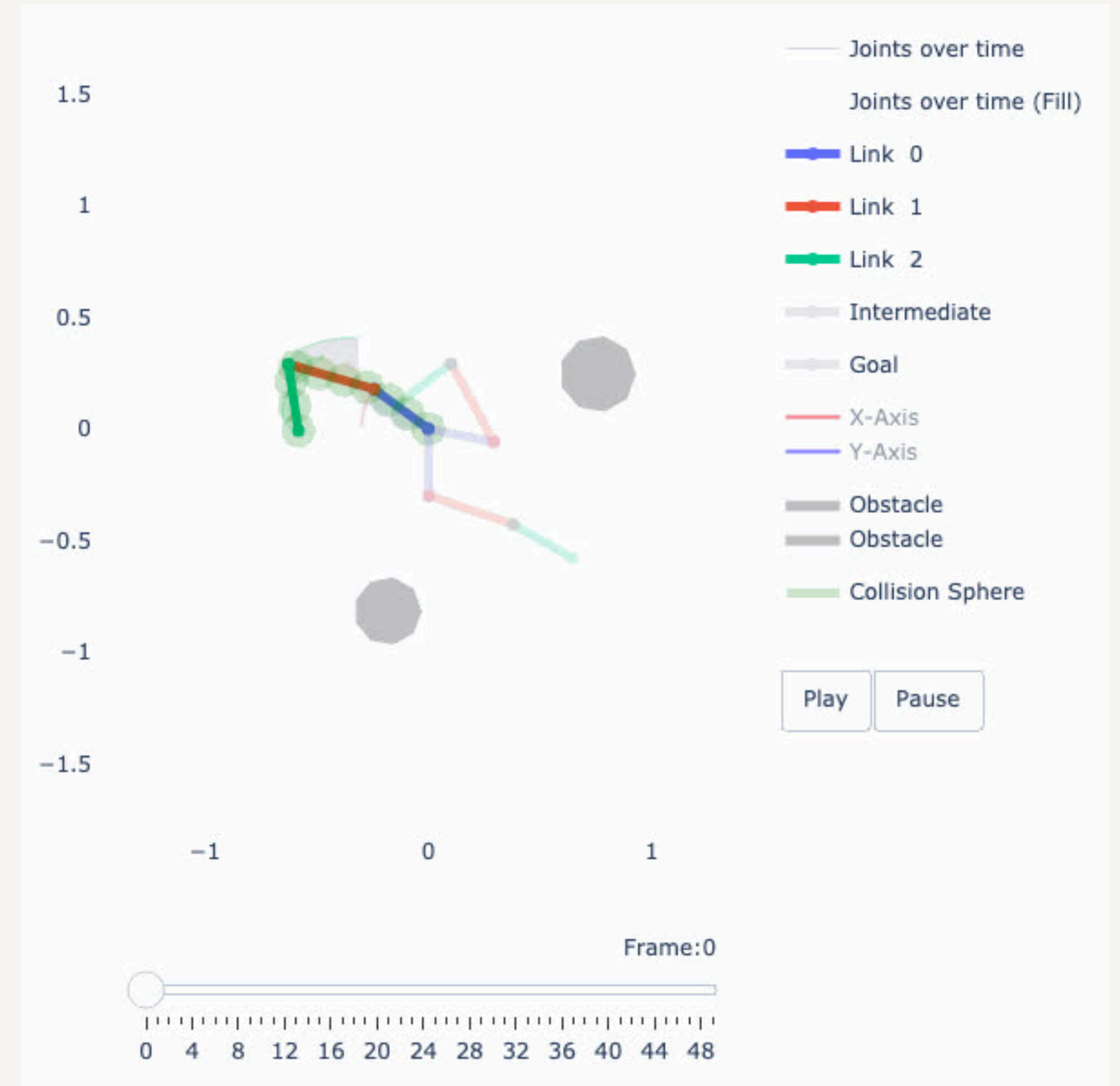
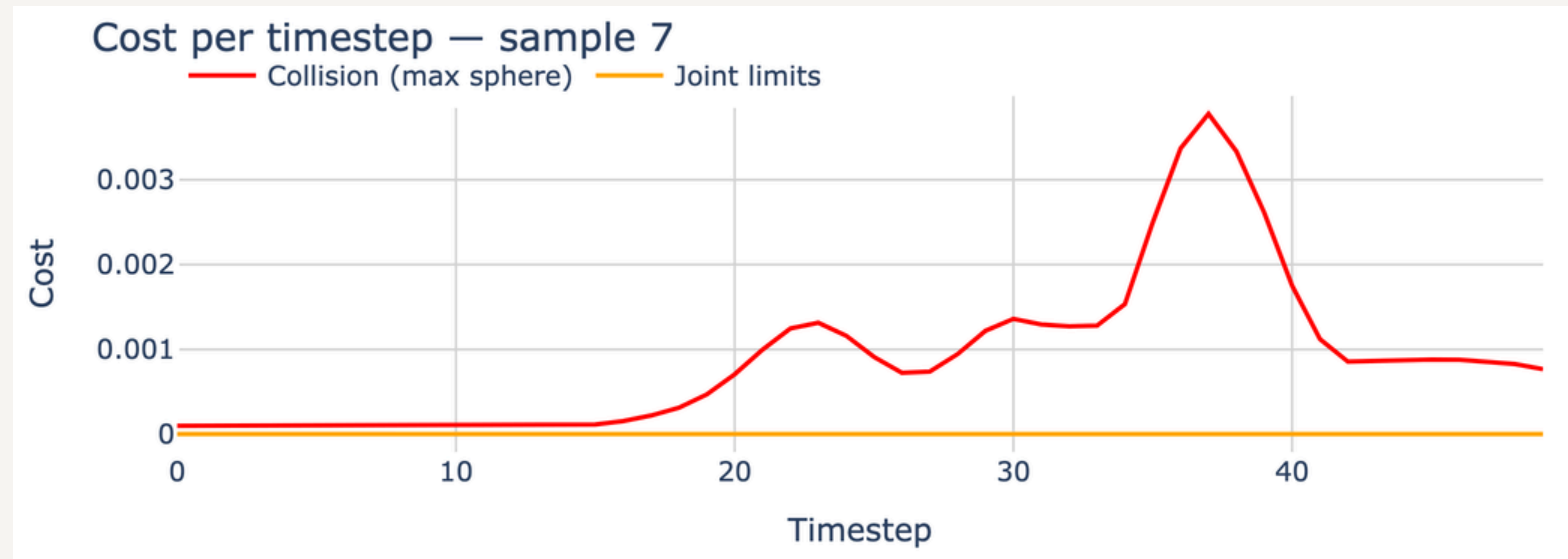


Experiments Evolution - 04

Trained with a bigger dataset & 1-2 obstacles per environments and validated on unseen data.

Problems to solve:

- Straight-line bias: optimizer follows the path of least resistance along the baseline (Only ONE baseline is computed).
- Shorter baseline distance $\neq$ collision-free path



Next Steps



- 01 Overcome straight-line bias in trajectory warm-starting through improved loss formulation and network architecture refinements.
- 02 Evaluate feasibility and quality of generated trajectories across diverse obstacle configurations and arm configurations.
- 03 Implement classical optimization-based planner for baseline comparison, evaluating accuracy, robustness and computation speed metrics.
- 04 Integrate gradient-based trajectory refinement; analyze convergence speed with and without learned warm-starts. Note: No change to initial research hypothesis.

Thank You For Your Attention

28.05.2026



Questions?

Topic #10 | Advanced Deep Learning for Robotics

Timo Matuszewski, Gabriel Salinas